FH University of Applied Sciences TECHNIKUM WIEN

Homework

es müssen nicht viele Features sein. Nächste Aufgabe wird weniger sein.

Design Patterns

Implement a software application that integrates three complementary design patterns from the Gang of Four

sich ergänzende

Overview

Build a small but production shaped prototype that integrates exactly three Gang of Four design patterns to solve a concrete problem, while explicitly addressing quality attributes: concurrency correctness, performance, and testability.

Objective

Select exactly 3 complementary GoF design patterns and integrate them in one application. Demonstrate that the patterns improve extensibility, separation of concerns, and runtime flexibility. Provide evidence of correctness under concurrency and evidence-based performance decisions.

Constraints

- GoF-only: your three patterns must be chosen from the 23 classic GoF patterns.
- No pattern libraries. Implement the patterns from scratch.
- Standard language/runtime libraries are allowed
- Non-GoF patterns may appear as architecture choices, but DO NOT count as your three patterns.

Technical Requirements

Design Patterns (GoF)

At least 1 must be Behavioural, and at least 1 must be Structural or Creational.

Patterns must be complementary: they should jointly enable a cleaner architecture than any pattern alone.

FH University of Applied Sciences TECHNIKUM WIEN

Concurrency + Correctness

multithreaded

thread pool zum bsp glaube ich

- Use a bounded concurrency model. Do not create unbounded threads.
- Include at least one shared resource (cache, registry, queue, repository, event bus, metrics) and define its thread-safety guarantees.
- At least one of your three GoF patterns must participate in concurrent execution OR be responsible for protecting shared state.

Concurrency Testing

Provide at least one test that can expose races, ordering bugs, deadlocks, or starvation. For example stress test or deterministic concurrency test.

Performance + Measurement Discipline

i agree for the measurement point x -> the request does not take longer than seconds

Define one hot path use case and a performance budget, latency and/or throughput.

Measure and report results with a reproducible methodology.

Provide a before/after comparison with at least one optimisation attempt and what you learned.

Testability

Provide a good coverage using automated tests. Both unit and integration tests should be present and clearly separated. **A 70% coverage should be reached.**

Architecture Communication

Include 2 Architecture Decision Records documenting major trade-offs. For example, why this concurrency model, why these patterns, why these performance metrics.

Deliverables

- README: problem statement, scope, constraints, quickstart (build/run/test/benchmark), pattern map, and interaction diagram.
- Source code in GitHub or FH GitLab (make sure it is accessible)
- Performance report (markdown is fine)
- Video (5 minutes)

Video

5 Minuten ca

- The problem + a quick demo.
- Architecture walkthrough + why these three GoF patterns.
- Concurrency model and how you tested correctness.
- Performance results, measurement methodology
- Lessons learned
- Bonus: how you leveraged AI as a developer tool

Grading

Category	Points
Correct GoF pattern implementations - **Complex patterns get more points**	25
Complementary integration of 3 patterns	15
Concurrency model + correctness	20
Performance	15
Testability and Coverage	15
Architecture	10

Suggested Project Shapes

Pick any domain you like, but choose a shape below that naturally requires concurrency, measurement, and tests.

Concurrent workflow runner

- Command: represent jobs/tasks as first-class objects.
- Template Method: workflow skeleton - validate, execute, persist, notify.
- Observer: job lifecycle events + metrics hooks.

Challenges: bounded worker pool, non-blocking observers, ordering/invariant tests.

Pluggable rules engine with instrumentation

- Strategy: runtime selection of evaluation algorithms.
- Abstract Factory or Factory Method: create rule sets from configuration.
- Decorator: caching/timing/metrics wrappers without changing core strategies.

Challenges: cache contention and correctness under concurrency; meaningful performance optimisations.

Scalable in-memory model + operations

- Flyweight: share immutable values to reduce allocations/memory footprint.
- Proxy: lazy-load or memoize expensive computations.
- Visitor: run multiple operations over a stable object structure - export, validate, stats.

Challenges: memoization correctness under concurrency; measurable allocation/GC improvements.

Event processing pipeline

- Chain of Responsibility: pipeline stages - validate, enrich, persist.
- Strategy: swap enrichment/persistence algorithms at runtime.
- Facade: stable API over internal subsystem wiring; simplifies testing and usage.

Challenges: step isolation tests; careful parallelism without breaking invariants.